University of Computer & Emerging Sciences
Karachi

Artificial Intelligence-School of Computing

Fall 2026, Lab Manual – 07s

Course Code:	Course: Computer Vision Lab
Instructor(s):	Talha Shahid

1. 2D Transformations

- 2D transformations change the position or shape of an image in the x and y directions.
  They include moving (translation), rotating, resizing (scaling), and tilting (shearing) an image.

2. 3D Transformations

- 3D transformations work with three axes – x, y, and z – adding depth to objects.
  They are used to move, rotate, or resize 3D objects like cubes or models in space.

3. Homogeneous Coordinates

- Homogeneous coordinates represent points with an extra dimension, like (x, y, 1).
  This makes all transformations (move, rotate, scale) possible using matrix multiplication.

4. Affine Transformations

- Affine transformations include translation, rotation, scaling, and shearing.
  They keep parallel lines parallel, so shapes like rectangles remain rectangles after transformation.

5. Projective Transformations / Homography

- Projective transformations (or homography) change the image's perspective.

They can make tilted images look straight — for example, turning a book photo taken from an angle into a top-down view.

Camera Calibration

Objective

- Steps for calibrating a pinhole camera model using a planar checkerboard (chessboard) pattern
- Intrinsic parameters
- Extrinsic parameters
- Distortion models
- Calibration equations
- Computing reprojection error
- Complete Python/OpenCV example

Required Materials

- A digital camera (webcam, phone, or industrial camera)
- A printed checkerboard (recommended inner corners 6×9 or 9×6)
- Tripod (optional but recommended)
- Computer with Python and OpenCV installed
- Sample images (at least 10–20 images from different viewpoints and distances)

Background Theory

1. Pinhole Camera Model

The pinhole camera models the mapping from a 3D world point $X_w=[X,Y,Z,1]^T$ to a 2D image point $x=[u,v,1]^T$ as:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Where:

- **λ\lambda** is a scale factor (depth)
- **K** is the intrinsic camera matrix
- **[R | t]** are extrinsic parameters that transform world coordinates to camera coordinates

The intrinsic matrix K is:

$$\mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- fx,fy are focal lengths in pixels (may differ if pixels not square)
- s is the skew (usually 0)
- (cx,cy) is the principal point (image center)

2. Extrinsic Parameters

Extrinsic parameters describe the rigid transform from world coordinates to camera coordinates:

$$\mathbf{X}_c = \mathbf{R}\mathbf{X}_w + \mathbf{t}$$

R is a 3×3 rotation matrix and **t** is a 3×1 translation vector.

3. Lens Distortion

Real lenses introduce distortion. Commonly used model in OpenCV includes radial and tangential distortion.

Radial distortion (approximation):

$$x_{distorted} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$
$$y_{distorted} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

Tangential distortion (decentering):

$$x' = x + [2p_1 xy + p_2(r^2 + 2x^2)]$$
$$y' = y + [p_1(r^2 + 2y^2) + 2p_2 xy]$$

OpenCV represents distortion as a vector: ***[k1,k2,p1,p2,k3]***.Some models extend to ***k4,k5,K6*** and thin prism / tilt parameters.

4. Calibration Goal

Given known 3D points $\mathbf{X}_w$ (checkerboard corners in the plane, Z=0) and their detected 2D image points x\mathbf{x}, estimate **K,R,t,** and distortion coefficients so that the projection error is minimized.

Mathematically: minimize the sum of squared projection errors over all points:

$$\min_{\mathbf{K},\mathbf{R}_i,\mathbf{t}_i,\mathbf{d}} \sum_i \sum_j \|\mathbf{x}_{ij} - \hat{\mathbf{x}}(\mathbf{K},\mathbf{d},\mathbf{R}_i,\mathbf{t}_i,\mathbf{X}_j)\|^2$$

Where ii indexes images, **j** indexes points, and **x^** is the projected 2D point under the current parameters.

5. Reprojection Error

After calibration, compute reprojection error for each detected point:

$$\epsilon = \sqrt{(u-\hat{u})^2 + (v-\hat{v})^2}$$

Report the mean reprojection error across all points. Typical good values depend on image resolution and detector accuracy — values below 0.5–1.0 pixels are usually considered good for standard setups.

Mathematical Summary (Matrices & Equations)

1. Intrinsic matrix **K**:

$$\mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

2. Full projection matrix (3×4):

$$\mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$$

3. Projection (homogeneous):

$$\tilde{\mathbf{x}} = \mathbf{P}\tilde{\mathbf{X}}_w$$

Where **·~ tilde** denotes homogeneous coordinates.

4. Distortion applied in normalized camera coordinates. Convert pixel coordinates to normalized by:

$$[x_n, y_n, 1]^T = \mathbf{K}^{-1}[u, v, 1]^T$$

Apply distortion to **(xn,yn),** then reproject to pixels by $K[x_{dist}, y_{dist}, 1]^T$

Lab Procedure

1. **Prepare the checkerboard**: Print a checkerboard with known square size. Record the number of inner corners (e.g., 6×9 means 6 along width and 9 along height).
2. **Capture images**:
   - Capture 15–30 images with different orientations and distances.
   - Ensure the checkerboard fills a variety of positions in the frame and is not always centered.
   - Avoid motion blur; use tripod if possible.
3. **Detect corners**: Use functions like cv2.findChessboardCorners to detect inner corners.
4. **Refine corners**: Use cv2.cornerSubPix to refine to sub-pixel accuracy.
5. **Create object points**: For each image, create the corresponding 3D points in the checkerboard coordinate system (Z = 0). For example, points (0,0,0), (1,0,0), (2,0,0), ... scaled by the square size.
6. **Calibrate**: Call cv2.calibrateCamera with object points and image points. This returns camera matrix, distortion coefficients, rotation and translation vectors for each image.
7. **Evaluate**: Compute reprojection error, inspect distortion coefficients, and optionally undistort sample images using cv2.undistort.
8. **Save parameters**: Save mtx, dist, rvecs, tvecs for later use.

```
import cv2
import numpy as np
import os
import glob

# Defining the dimensions of checkerboard
CHECKERBOARD = (6, 9)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# Creating vector to store vectors of 3D points for each checkerboard image
objpoints = []
# Creating vector to store vectors of 2D points for each checkerboard image
imgpoints = []
```

```
# Defining the world coordinates for 3D points
objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
objp[0, :, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape(-
1, 2)
prev_img_shape = None

# Extracting path of individual image stored in a given directory
images = glob.glob('./images/*.jpg')
for fname in images:
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Find the chess board corners
    # If desired number of corners are found in the image then ret = true
    ret, corners = cv2.findChessboardCorners(gray, CHECKERBOARD,
cv2.CALIB_CB_ADAPTIVE_THRESH +
                                             cv2.CALIB_CB_FAST_CHECK +
cv2.CALIB_CB_NORMALIZE_IMAGE)

    """
    If desired number of corner are detected,
    we refine the pixel coordinates and display
    them on the images of checker board
    """
    if ret == True:
        objpoints.append(objp)
        # refining pixel coordinates for given 2d points.
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
criteria)

        imgpoints.append(corners2)

        # Draw and display the corners
        img = cv2.drawChessboardCorners(img, CHECKERBOARD, corners2, ret)

    cv2.imshow('img', img)
    cv2.waitKey(0)

cv2.destroyAllWindows()

h, w = img.shape[:2]

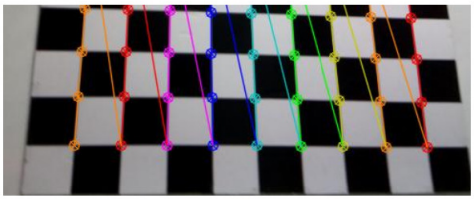
"""
Performing camera calibration by
passing the value of known 3D points (objpoints)
and corresponding pixel coordinates of the
detected corners (imgpoints)
"""
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints,
gray.shape[::-1], None, None)


print("========================= Camera matrix ==========================")
```

```
print(mtx)

print("======================== Distortion ==========================")
print(dist)
print("======================== Rotation Vector ========================")
print(rvecs)
print("======================== Translation Vector
========================")
print("tvecs : \n")
print(tvecs)
```

Output



```
[[ 2.11014320e-01 -4.84232133e-01  3.54626679
   2.58120491e-01]]
======================== Rotation Vector ====
(array([[-0.31041628],
       [ 0.15247297],
       [ 1.57728648]]), array([[0.31415841],
```

Perspective Projection:

Perspective Projection is the mathematical model that describes how three-dimensional (3D) objects in the real world are transformed into a two-dimensional (2D) image, mimicking the process of a **pinhole camera** or the human eye. It is the most common model used in computer vision and graphics because it accurately captures the phenomenon of **perspective**, where objects appear smaller as they move farther away

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread('monalisa.jpg')

h, w = img.shape[:2]

# --- Transformation Function ---
```

```
def apply_perspective_transform(original_img, h_dim, w_dim, source_points, dest_points, title):

# 1. Calculate the Homography Matrix (M)

M = cv2.getPerspectiveTransform(source_points, dest_points)
```

```
# 2. Apply the Perspective Transformation

img_transformed = cv2.warpPerspective(original_img, M, (w_dim, h_dim))
```

```
# 3. Display Results (simplified for Word copy-paste)

cv2.imwrite('transformed_image.jpg', img_transformed)

print("---", title, "---")

print("Homography Matrix (M) Calculated.")

print("Transformed image saved as 'transformed_image.jpg'")

# In a Colab environment, use Matplotlib/cv2_imshow for viewing
```

```
# --- 3. Define Points for Angle Change (Example: Tilted from Top) ---

# Source Points (The four corners of the original image)

pts1 = np.float32([

[0, 0],           # Top-Left

[w - 1, 0],       # Top-Right

[w - 1, h - 1],   # Bottom-Right

[0, h - 1]        # Bottom-Left

])

# Destination Points (Simulate a strong tilt, top edge is compressed/farther away)

offset_x = w // 5
```

```
offset_y = h // 4

pts2_tilted = np.float32([

[offset_x, offset_y],           # New Top-Left (Moved inward
and down)

[w - 1 - offset_x, offset_y],   # New Top-Right (Moved inward
and down)

[w - 1, h - 1],                 # Bottom-Right (Stays
wide/close)

[0, h - 1]                      # Bottom-Left (Stays
wide/close)

])

# --- Execute ---

apply_perspective_transform(img, h, w, pts1, pts2_tilted,
"Tilted View from Below")
```

COLMAP

COLMAP is a general-purpose Structure-from-Motion (SfM) and Multi-View Stereo (MVS) pipeline with a graphical and command-line interface. It offers a wide range of features for reconstruction of ordered and unordered image collections.

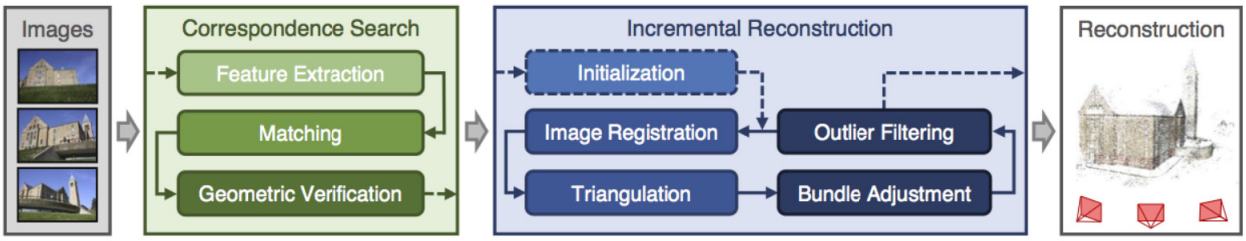
The latest source code is available at GitHub. COLMAP builds on top of existing works and when using specific algorithms within COLMAP

Executables and other resources can be downloaded from https://demuc.de/colmap/.

Getting Started

1. Download the pre-built binaries or build the library manually from source.
2. Download one of the provided datasets (see Datasets) or use your own images.
3. Use the **automatic reconstruction** to easily build models with a single click.

Structure-from-Motion



COLMAP's incremental Structure-from-Motion pipeline.

Structure-from-Motion (SfM) is the process of reconstructing 3D structure from its projections into a series of images. The input is a set of overlapping images of the same object, taken from different viewpoints. The output is a 3-D reconstruction of the object, and the reconstructed intrinsic and extrinsic camera parameters of all images. Typically, Structure-from-Motion systems divide this process into three stages:

1. Feature detection and extraction
2. Feature matching and geometric verification
3. Structure and motion reconstruction

COLMAP reflects these stages in different modules, that can be combined depending on the application.